

Informe de diseño — Entrega 1

Distribuidora 4 · IIC3103 Taller de Integración 2026-2

Documento para Canvas. Las secciones A-D y la declaración de IA corresponden al PDF *Proyecto - Entrega 1*. Los diagramas de arquitectura, integración y ER son anexos.

Fuente en el repositorio: `docs/informe-e1/`.

Informe de diseño — Entrega 1, Distribuidora 4

IIC3103 Taller de Integración, 2026-2. Grupo 4.

Este directorio es el **informe de diseño** que pide el PDF de Entrega 1 (§12.1 / forma de entrega §17). Se sube a Canvas como PDF (`INFORME.md` es el cuerpo concatenado). El código vive en el repositorio GitHub asignado.

El enunciado (`docs/01_enunciado_proyecto_llm.md` , fuente: *Proyecto - Entrega 1.pdf*) manda el diseño. `docs/02_arquitectura_decisiones_entrega1.md` es una **propuesta inicial** (Next.js, esquema en inglés) que **no describe lo que corre**. Aquí se documenta el sistema desplegado: FastAPI + worker + Postgres + SPA Vite.

Mapa al PDF

Ítem del PDF	Archivo
A. Diagramas de secuencia (prosa + figuras PNG)	01-diagramas.md
Diagramas Mermaid a tamaño real (mejor lectura)	diagramas.md
Anexos: arquitectura, integración, ER	01-diagramas.md §§4-6 y diagramas.md
B. Tres decisiones + trade-offs y arrastre a E2/E3	02-decisiones-y-proyeccion.md
C. Estrategia de abastecimiento y producción	03-estrategia-abastecimiento.md
D. Modelo de custodia y trazabilidad	04-modelo-custodia.md

Ítem del PDF	Archivo
Servidor, HTTPS, 30 kits, huecos logísticos	05-logistica-operacion.md
Declaración de uso de IA (§18)	06-declaracion-ia.md y <code>README.md</code> en la raíz del proyecto
Cuerpo único (Markdown)	INFORME.md
HTML (diagramas renderizados)	INFORME.html
PDF para Canvas	Informe_Entrega1_Distribuidora4.pdf

Los diagramas de secuencia no son genéricos: usan SKU, cantidades y solicitudes que quedaron en `logs.txt` de la corrida PROD del 6-7 de octubre de 2026 (30 unidades de cada kit) y una venta real de `KIT-RESP-ADULTO`.

Para regenerar figuras y PDF: `python3 _render_diagrams.py`, capturar `img/*.html` a PNG, `python3 _concat_informe.py && python3 _build_html.py`. El archivo `Informe_Entrega1_Distribuidora4.pdf` es el que se sube a Canvas. Los Mermaid en vivo están en `diagramas.md`.

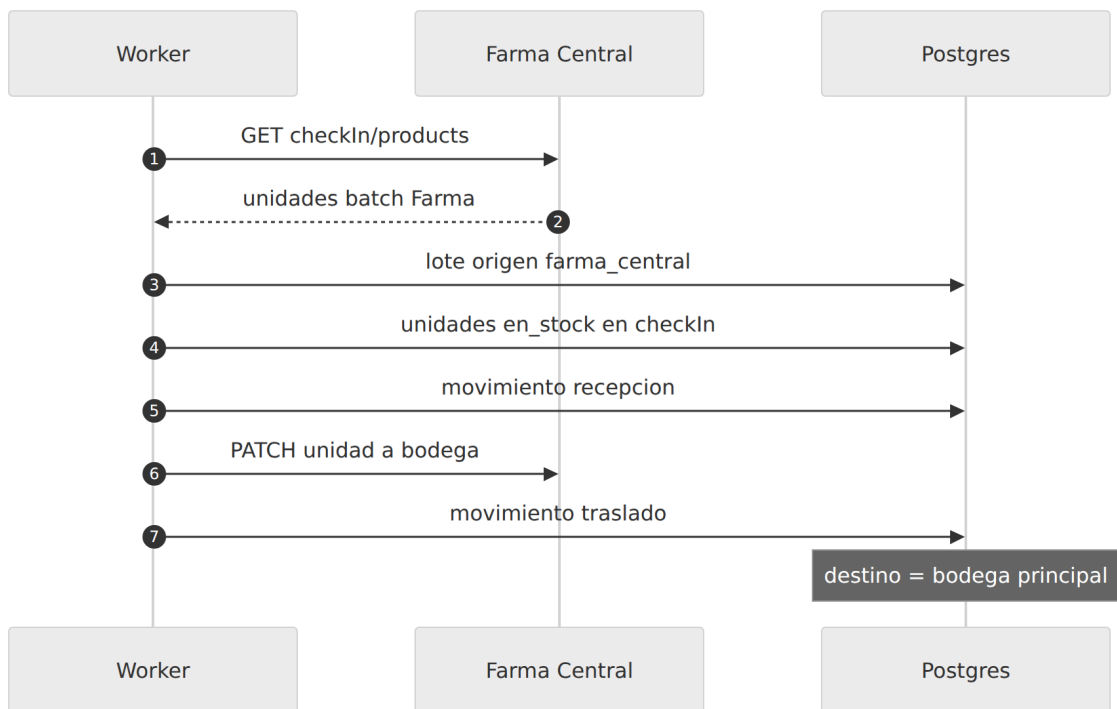
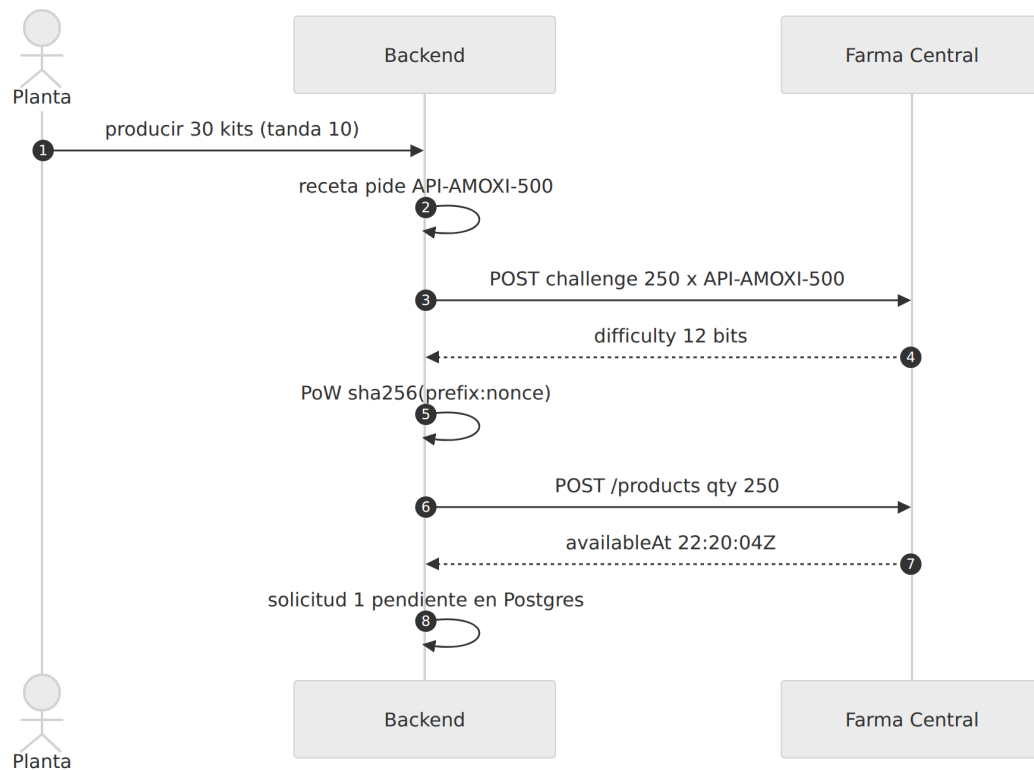
A. Diagramas de secuencia (e integración)

Los tres flujos del PDF están **instanciados** sobre `KIT-RESP-ADULTO` y su cadena de amoxicilina. Fuente visual (Mermaid a tamaño real, se lee mejor en GitHub/Cursor): [diagramas.md](#). Aquí va la prosa y, para el PDF, las mismas figuras exportadas a PNG.

Cadenas del repo: `comprar` → `_pedir_con_pow` → `request_products` → `registrar_compra`; `fabricar` → `registrar_fabricacion`; `iniciar_checkout` → `crear_transaccion` → `finalizar`.

1. Abastecimiento de `API-AMOXI-500`

Instancia PROD (6 oct 2026, `logs.txt`): `producir` arma 30 kits de cada tipo. Primera compra: **solicitud #1, 250 × `API-AMOXI-500`**. PoW 12 bits. Farma responde `availableAt 22:20:04Z (~31 s)`. El lote nace en `checkIn` y el worker lo deja en **bodega** (el API no es frío). 250 es múltiplo del `production.batch` de esa noche.



Código: `app/operaciones.py` (`comprar` , `_pedir_con_pow`), `app/pow.py` , `app/farma_client.py` , `app/custodia.py` (`registrar_compra` , `registrar_llegadas`), `worker/main.py` .

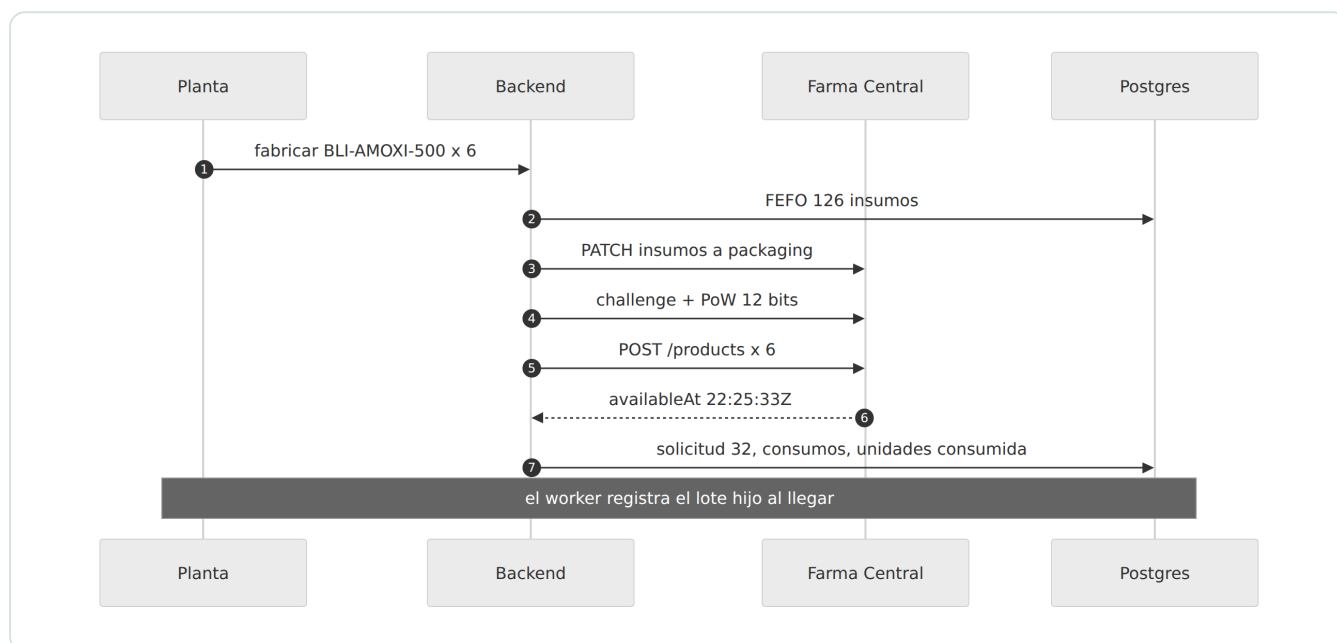
2. Acondicionamiento: BLI-AMOXI-500 y armado de KIT-RESP-ADULTO

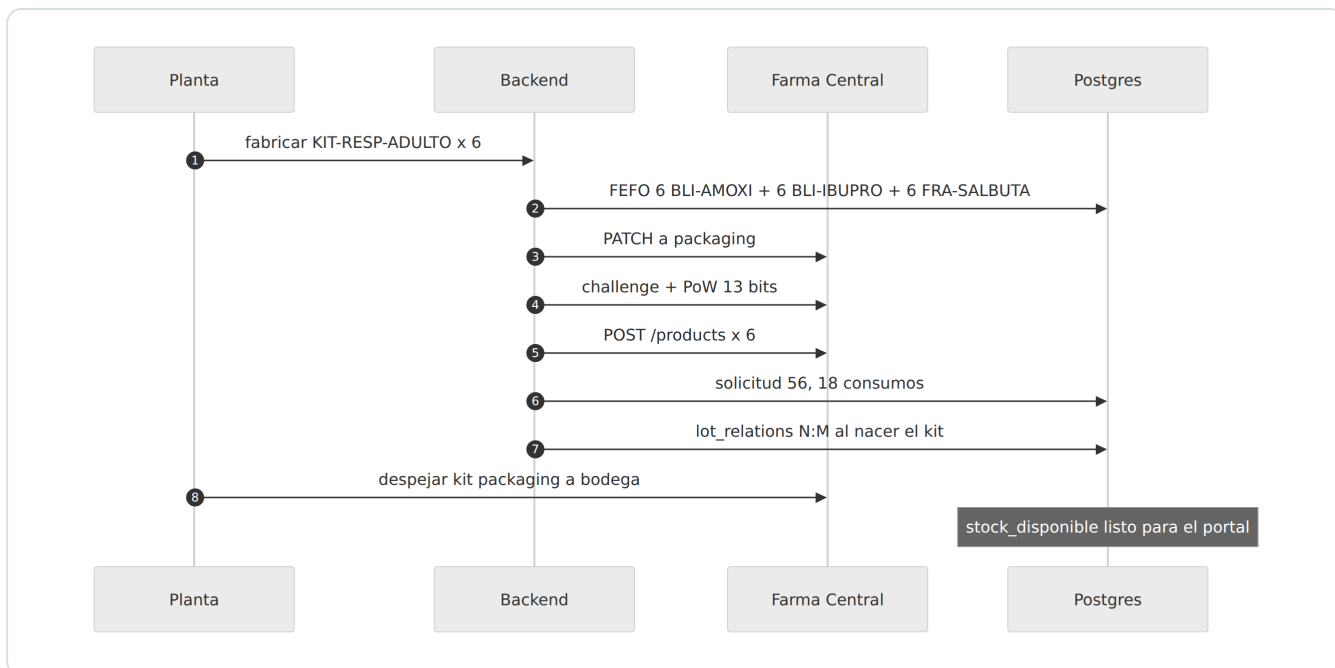
El PDF pide **los dos tramos**. Receta del enunciado (§4 y §8.2):

Componente	Por 1 × BLI	Lote de 3	Lote de 6 (PROD #32)
API-AMOXI-500	12	36	72
EXC-LACTOSA-DC	8	24	48
LAM-BLISTER-PVC	1	3	6
Unidades consumidas	21	63	126

Tramo 1: fabricación #32, 6 × BLI-AMOXI-500, 126 insumos, PoW 12 bits. FEFO → packaging → challenge → consumo. El worker registra la **generación** del lote hijo.

Tramo 2: fabricación #56, 6 × KIT-RESP-ADULTO, 18 acondicionados (6+6+6). PoW 13 bits. El kit ambiente se despeja a bodega.

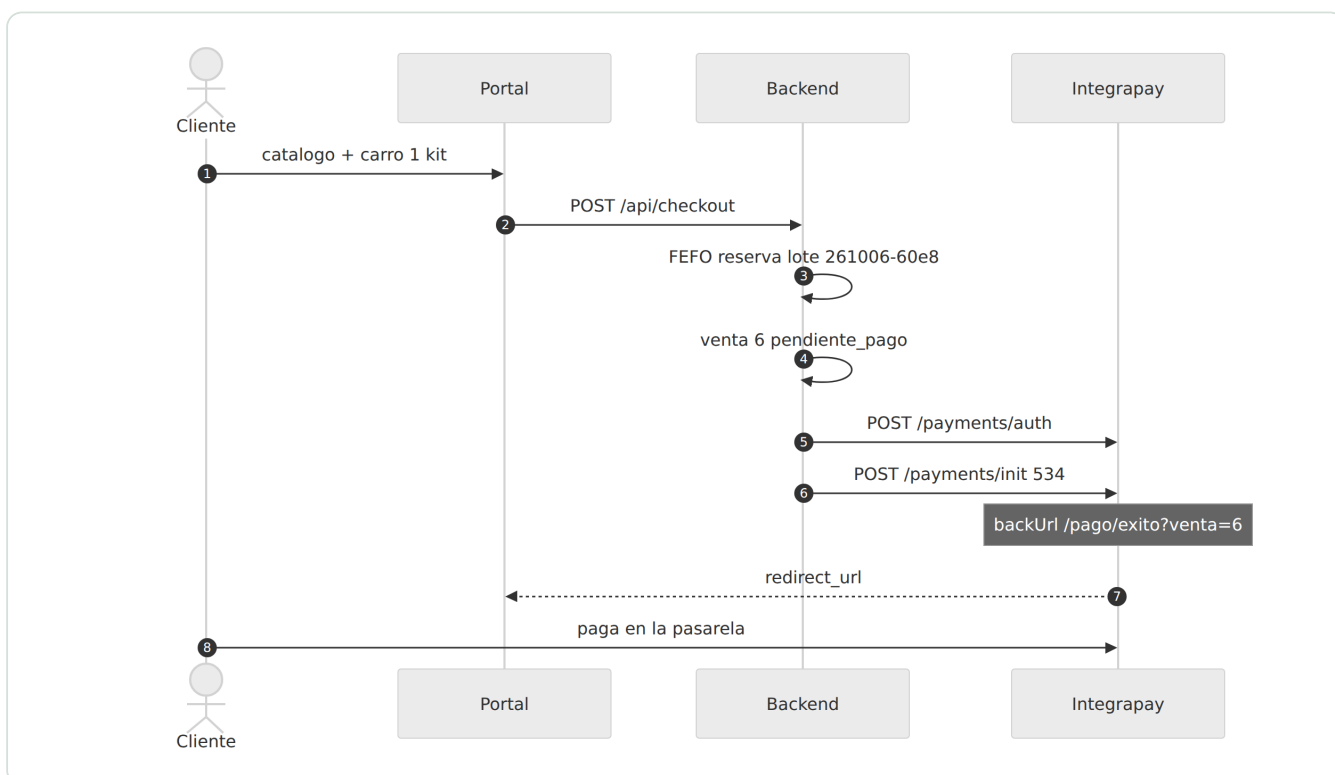


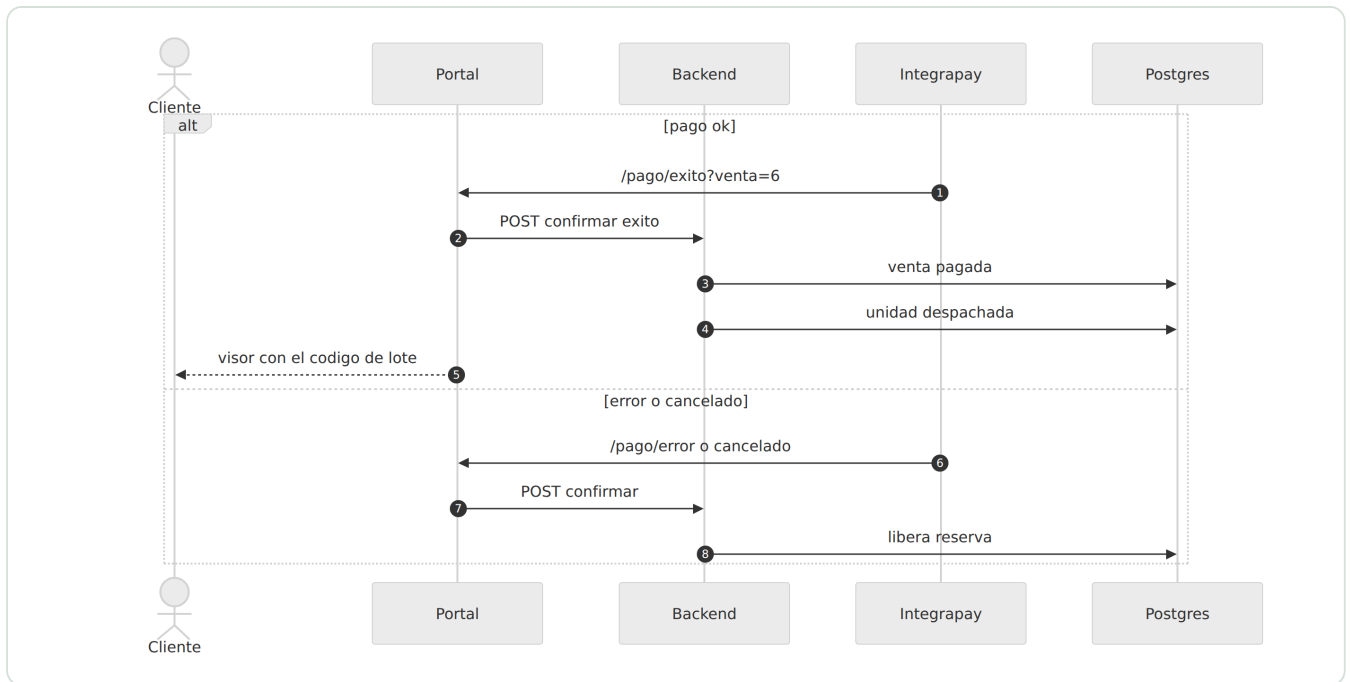


`fabricar` no espera el `availableAt`. Si Farma acepta el POST y Postgres falla, queda un `CRITICAL` con los `_id` (no hay transacción distribuida).

3. Venta de KIT-RESP-ADULTO

Instancia PROD: pedido #6, `pagada`, 1 kit, lote FEFO `L-KIT-RESP-ADULTO-261006-60e8`, \$534. Integrpay real (~20 % de error en E1). `DISPATCH_FARMA=0`: despacho solo en custodia, sin `PATCH` a `checkOut`.

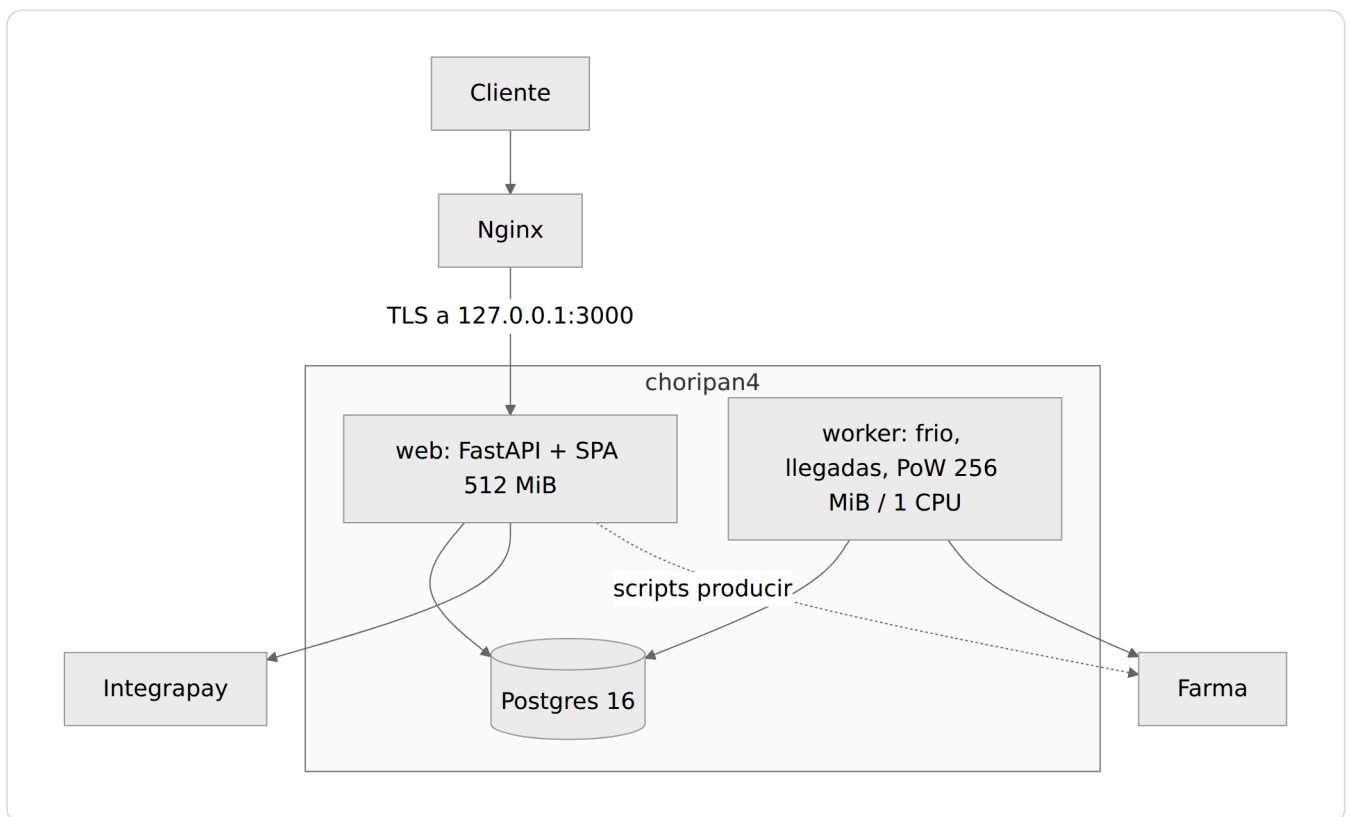




Código: `web/src/pages/Checkout.tsx` , `app/ventas.py` , `app/integrations/checkout.py` , `app/main.py` .

4. Anexo — Arquitectura de procesos

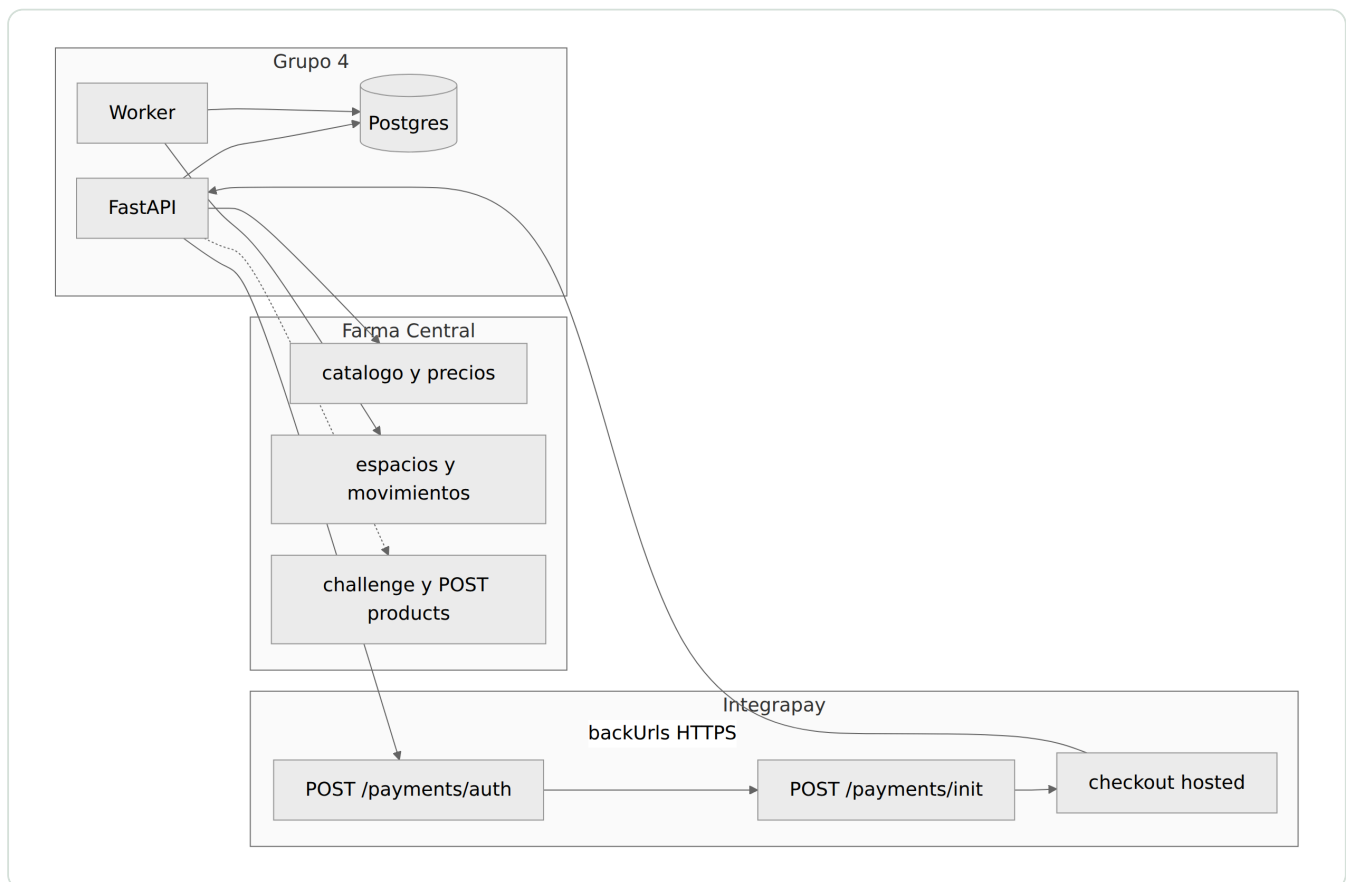
No lo exige el PDF como diagrama nombrado. Partición en el servidor de 2 GB.



Unidad	Qué hace	Límite
web	Uvicorn, SPA <code>web/dist</code> , APIs	512 MiB, 2 workers
worker	Llegadas, frío, <code>recuperar_pendientes</code> ; PoW en worker o <code>compose run</code>	256 MiB, 1.0 CPU
db	Custodia	512 MiB, puerto <code>127.0.0.1:5433</code>

Nginx termina TLS hacia `127.0.0.1:3000` . **Nunca** `docker compose down -v` .

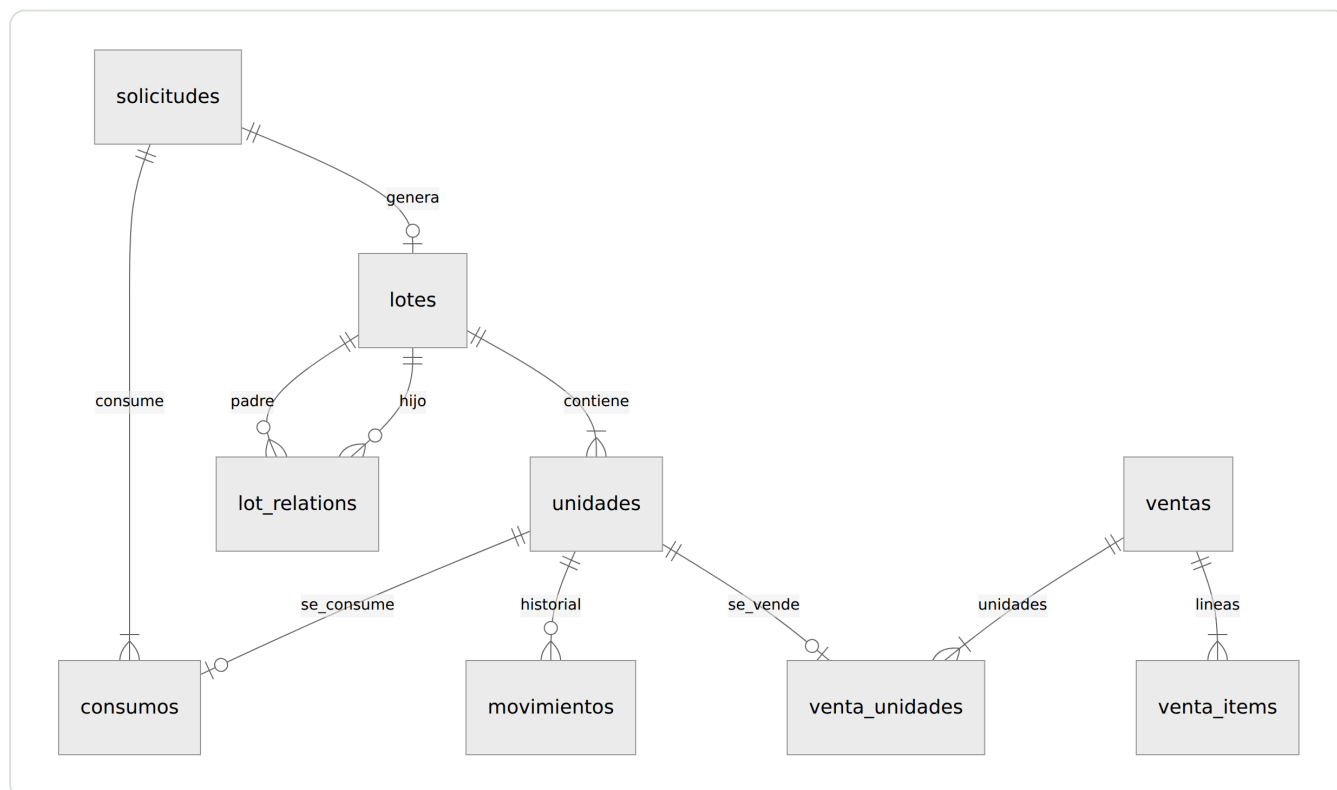
5. Anexo — Integración con sistemas del curso



Contraparte	Contrato	Uso en E1
Farma Central	<code>farma_client.request</code> (token, 401, 429)	Catálogo, precios, espacios, PoW, compras, fabricaciones, movimientos
Integrapay	<code>POST /payments/auth</code> + <code>POST /payments/init</code> y <code>backUrls</code>	Host prod sin sufijo <code>/checkout</code>
Visor	No llama a Farma	Postgres, UI en <code>/trazabilidad? lote=</code>

6. Anexo — Modelo de datos (ER)

Tablas en `app/models.py`. Nombres en español.



Relación consumido↔generado **N:M**, materializada en `lot_relations` al nacer el hijo.

B. Tres decisiones de arquitectura y proyección E2/E3

El PDF pide tres decisiones con qué / por qué / qué se gana / qué se pierde. Las que siguen son las que **quedaron en el código**, no las de la propuesta 02 (Next.js, backend TypeScript, tablas `lots` / `custody_events`).

El enunciado avisa que las decisiones de E1 se arrastran el semestre. Cada decisión cierra con lo que deja listo o explícitamente aplazado para E2/E3.

Decisión 1 — Un runtime Python y un portal estático (no Next.js)

Qué se decidió. Conservar FastAPI + SQLAlchemy + Alembic. El portal es una SPA Vite/React/TypeScript que el Dockerfile construye a `web/dist` y FastAPI sirve con `StaticFiles`. Un solo proceso HTTP (`web`) más un `worker`. No hay servidor Node en producción.

Por qué. El compañero ya tenía Farma, PoW, espacios y el esqueleto de custodia en Python. El servidor son 2 núcleos y 2 GB. Levantar Next.js junto a Uvicorn duplica runtime, RAM y puertos. Migrar el backend a TypeScript en la semana de E1 rompía integraciones que ya hablaban con Farma.

Qué se gana. Un deploy (`docker compose build web`), un puerto detrás de Nginx (`127.0.0.1:3000`), el PoW no pelea con el event loop del portal si se corre en el worker o en `compose run`. El catch-all `GET /{full_path:path}` entrega la SPA en `/`, `/trazabilidad`, `/pedidos`, `/pago/...`.

Qué se pierde. No hay SSR ni React Server Components. El carrito vive en `localStorage` (`web/src/cart.ts`), no en sesión de servidor. El build de frontend queda acoplado a la imagen Python. Quien esperaba el árbol de rutas de Next tiene que leer `web/src/App.tsx`.

Trade-off. Preferimos no reescribir lo que ya compraba insumos, a cambio de un frontend más “clásico”. Si en E3 hiciera falta un canal con UI pesada, se puede volver a separar el frontend; el API ya está detrás de `/api/...`.

Arrastre. E2 pide órdenes de compra y lenguaje natural: esos canales entran como nuevas rutas o un adapter que llama `custodia.crear_venta(..., canal="oc"|"nl")`, no como un rewrite de framework.

Decisión 2 — Postgres del grupo como fuente de verdad de custodia

Qué se decidió. Farma ejecuta el mundo físico (unidades, espacios, fabricación). El historial interno, la genealogía y el stock que muestra el portal salen de Postgres (`unidades.estado`, `lotes`, `movimientos`, `lot_relations`, `ventas`). El visor **no consulta Farma**.

Por qué. El enunciado §8.6 es explícito: Farma no guarda el historial del grupo. Un visor que recorriera `GET /spaces/.../products` en cada búsqueda no escala (el PDF lo dice: no cargar todo el historial) y no respondería aguas arriba/abajo ni clientes.

Qué se gana. Consultas del visor en el orden de decenas de milisegundos (CTE sobre `lot_relations` , profundidad máxima 20). Stock del catálogo = `custodia.stock_disponible` (en_stock, no vencido, no reservado). Independencia si Farma está lenta al mirar un lote.

Qué se pierde. Hay que **sincronizar**. El worker llama `registrar_llegadas` cada 20 s; si Farma entrega una unidad y el tick falla, hay una ventana en que el físico existe y custodia no. No usamos two-phase commit. El despacho a Farma (`DISPATCH_FARMA`) está apagado en E1 para no divergir: la unidad se marca `despachada` en Postgres y sigue en el espacio Farma.

Trade-off. Consistencia eventual de llegadas a cambio de un modelo consultable y de no mentir en el portal con el inventario crudo de Farma (que incluye vencidos y cosas en tránsito).

Arrastre. Recall E2: `origen` , `origen_grupo` , `cuarentena` ya están en el enum de espacios y en `estado` de unidad. Falta el job que, dado un código de lote Farma, recorra `aguas_abajo` , bloquee venta y mueva a `quarantine` . El grafo ya está.

Decisión 3 — Portal síncrono, planta asíncrona (worker + candados)

Qué se decidió. El click de “Pagar” es un request HTTP que reserva FEFO, llama Integrapay y redirige. Comprar insumos, fabricar, PoW, cadena de frío y reintento de despachos corren en el **worker** (o en un proceso `producir` aparte), con `os.nice(10)` y `cpus: "1.0"` . Fabricación y venta usan advisory locks de Postgres (`LOCK_FABRICACION` , `LOCK_VENTA`) para no elegir las mismas unidades en paralelo.

Por qué. El Anexo de PoW y el propio PDF advierten que un solver greedy puede comerse la CPU del portal. La cámara de frío degrada producto si no se mueve en minutos (`TICK_SECONDS = 20`). Farma limita 250 req/min: el worker topea `MAX_MOVES_PER_TICK = 100` .

Qué se gana. El catálogo y el visor siguen respondiendo mientras se resuelve un challenge de 12-13 bits. La recepción no bloquea a `comprar` : se registra `solicitud` pendiente y el tick crea el lote cuando aparece el `batch` .

Qué se pierde. Hay latencia de un tick (~20 s) entre que Farma deja el producto y que custodia lo ve. El orquestador `producir(..., tanda=10)` es un bucle con `sleep(20)` , no una cola con DLQ. No hay outbox ni message broker (la propuesta 02 los mencionaba; en 2 GB no se justificó).

Trade-off. Operación continua barata versus menos “mensajería empresarial”. E2 puede enchufar una cola **como canal de pedidos** (`Venta.canal`) sin cambiar el worker de frío.

Arrastre. Operación 24-48 h (E2) y 48-72 h (E3): el worker ya reintenta recuperar_pendientes y no se mata por una excepción en el tick. Falta marcar vencidas en el mismo loop (registrar_vencidas existe, el worker aún no lo llama: hueco P2).

Otras decisiones menores que el informe debe poder defender

Genealogía duplicada a propósito. consumos guarda la unidad exacta. lot_relations agrega lote↔lote N:M para el visor. Coste: dos escrituras al nacer un lote. Beneficio: el CTE no recorre consumos × unidades en cada búsqueda.

SPA híbrida, no plantillas Jinja. El HTML de fallback del catch-all existe por si web/dist no está; en PROD el visor es React.

Confirmar pago sin sesión de usuario. POST /api/checkout/confirmar acepta venta_id + resultado porque Integrapay redirige al browser. Es el camino de las backUrls. Quien conozca un venta_id pendiente podría confirmar; en E1 el riesgo se acepta (grupo cerrado, HTTPS). E2 debería firmar las backUrls o validar el payment_id contra Integrapay.

Adapter de Integrapay, no de /transactions. El host real es ../payments/auth e init. Un base URL con sufijo /checkout devolvió 500 y dejó una reserva (venta 1, luego cancelada). El adapter ahora revierte la reserva si init falla.

Proyección a Entrega 2 y 3

El PDF recomienda diseñar E1 para no rehacer. Lo que ya está en el esquema y lo que falta:

Tema E2/E3	Qué hay hoy	Qué falta
Órdenes de compra / plazos	Venta.canal , orden_externa_id	Parser del canal, estados de plazo
Pedidos en lenguaje natural	Idem canal	Servicio no determinista; misma reserva FEFO
Colas / archivos	Campo canal	Adapter de ingestión; no hace falta un broker en el mismo host
Recall / retiro	aguas_abajo , clientes_del_lote , espacio quarantine , estado cuarentena	Job de bloqueo + notificación en ventana de tiempo

Tema E2/E3	Qué hay hoy	Qué falta
Facturación	<code>venta_items</code> con precio vigente al comprar	Adapter del sistema de facturas
Escasez / trueque entre grupos	<code>lotes.origen</code> $\in$ { <code>farma_central</code> , <code>distribuidora</code> , <code>produccion</code> }, <code>origen_grupo</code>	Registrar lotes que llegan de otra dist.
Cuarentena	Enum listo, worker no mira ese espacio en llegadas (a propósito)	Usar el espacio cuando Farma emita el retiro
Visor a escala	CTE acotado a un código, no dump de <code>movimientos</code>	Índices extra si el grafo crece en E3
Autonomía 24-72 h	Worker no muere por un tick fallido; candados	Job de vencimiento; alerta si <code>en_camino</code> se pudre

La decisión de **no** despachar en Farma en E1 (`DISPATCH_FARMA=0`) se revierte cuando el grupo acepta el riesgo de un `PATCH` a `checkOut` a medias: `despachos_pendientes` + `recuperar_pendientes` existen para ese día.

C. Estrategia de abastecimiento y producción

El PDF pide criterios **concretos**, no una declaración de intenciones. Los números salen de `app/scripts/calcular_requerimientos.py` (objetivo por defecto: 30 por kit), `app/produccion.py` (`tanda=10`) y la corrida PROD de `logs.txt` .

Esta política la ejecuta **quien corre** `producir` , no un cron automático. Eso es deliberado en E1: un job que compre solo contra precio volátil puede vaciar presupuesto o llenar `buffer` . Queda como hueco a automatizar en E2 si el curso lo exige.

Objetivo de planta (E1)

Antes de la fecha de entrega, en **PROD**, acondicionar **al menos 30 unidades de cada kit clínico vendible**. No hace falta que sigan en stock al evaluar; hace falta haberlas producido.

La corrida del 6 de octubre arrancó con:

```
Producir {KIT-RESP-ADULTO: 30, KIT-PEDIA-FEBRIL: 30, KIT-CURACION: 30,  
KIT-CARDIO-BASICO: 30, KIT-DIABETES: 30, KIT-ANALGESIA: 30,  
KIT-ATB-IV: 30, KIT-VACUNA-INFLU: 30, KIT-GASTRO: 30, KIT-DERMATO: 30}
```

Diez SKU × 30 = 300 kits pedidos. El orquestador redondea cada nivel al `production.batch` de Farma.

Umbrales de reposición

Nivel	Se dispara cuando	Acción
Kit	<code>stock_disponible + fabricaciones_en_camino</code> del SKU < una tanda (10) y el objetivo de campaña no está cubierto	<code>fabricar</code> ese kit por el mínimo múltiplo de batch que no pase lo que cabe en <code>packaging</code>
Intermedio (BLI, FRA, ...)	La receta del kit en tanda pide más de lo que hay en espacios usables	<code>fabricar</code> el intermedio, de mayor profundidad a menor
Insumo (API, EXC, LAM, ...)	<code>planificar</code> indica demanda y no hay suficiente <code>en_camino</code>	<code>comprar</code> el múltiplo de batch

`stock_disponible` ignora `reservada`, `despachada`, `consumida`, `vencida` y lo que vence en menos de 1 h.

No reponemos “al 20% de capacidad de bodega”. Reponemos **por tanda de kits**, porque el cuello de botella real es `packaging` (150 unidades de huella; `calcular_requerimientos` avisa si un lote no cabe).

Cantidades de compra

Siempre múltiplo de `tamano_lote(sku)` (`catalogo()[sku]["production"]["batch"]`). `comprar` rechaza cualquier otra cantidad.

Ejemplos de la primera ola PROD (`logs.txt`):

SKU	Cantidad comprada (solicitud)	Notas
<code>API-AMOXI-500</code>	250 (#1)	Insumo del blíster del kit respiratorio
<code>API-IBUPRO-400</code>	150 (#2)	Compartido con otros kits

SKU	Cantidad comprada (solicitud)	Notas
Más tarde API-AMOXI-500	100 (#71)	Segunda ola cuando la tanda siguiente pidió más

`calcular_requerimientos` explota la receta de **todos** los kits a la vez: un excipiente compartido se compra una sola vez, redondeado al batch, no diez veces. Eso importa: el enunciado dice que los insumos compartidos son intencionales.

La tanda de fabricación de kits no es 30 de un golpe: `producir(..., tanda=10)` arma de a 10 para no saturar acondicionamiento. Farma, en PROD, entregó lotes de kit de 3 o 6 unidades (p. ej. #56: 6 × `KIT-RESP-ADULTO`), que son múltiplos de su batch.

Bodega externa (`buffer`)

Farma cobra por hora en el espacio `buffer`. Criterio:

- **Productos de ambiente:** no se dejan en `buffer`. `producir` los despeja a `bodega`. Si el orquestador ve acondicionamiento lleno de intermedios que no avanzan, los vacía a bodega (`_vaciar_acondicionamiento`), no al buffer.
- **Productos de frío:** el worker mueve `checkIn / bodega` → `cold` si hay cupo; **solo si** `libres("cold") == 0` usa `buffer`. Al nacer en `packaging`, lo mismo (`nacidas_en_acondicionamiento`).
- **Aceptar el costo:** únicamente como válvula de la cámara, nunca como almacén de campaña. Una tanda extra “porque el precio bajó” **no** se autoriza si eso implica pagar buffer.

Capacidades: se leen de `GET /farma-central/spaces` (`app/espacios.py`). No se hardcodean IDs (cambian entre DEV y PROD).

Vencimiento y merma

- **Orden de consumo:** FEFO. `unidades_disponibles` ordena por `packaging` primero (ya están en la mesa) y después `vence_en` ascendente. Margen: no se usa lo que vence en **menos de 1 hora** (`timedelta(hours=1)`).
- **Portal:** `stock_disponible` usa el mismo margen. El catálogo no ofrece vencidos ni reservados.
- **Worker:** aún **no** llama `registrar_vencidas` cuando Farma desaparece unidades. El helper existe. Hasta que se enganche, un vencido puede seguir en Postgres como `en_stock` un tick más: el margen de 1 h cubre parte de esa ventana. Hueco declarado, no escondido.

- **Frío:** sacar un SKU frío de cámara degradada `expiresAt` . Por eso las frías se mueven a `packaging` al final de `fabricar` , y si el challenge falla se `_devolver_al_frio` .

Sensibilidad al precio ($\pm 50\%$)

El precio de venta del portal es el de `GET /farma-central/market/prices` en el momento de `validar_carrito` / checkout, no un valor fijo. El **costo** de insumo es el de Farma al comprar.

Shock	Política concreta
Precio de venta del kit +50 %	No adelantar tandas. Seguir el umbral de 10. El margen mejora; producir de más inmoviliza <code>packaging</code> y riesgo de merma.
Precio de venta -50 %	Una tanda adicional (10 kits, redondeada a batch) solo si <code>libres("bodega")</code> o <code>libres("cold")</code> cubren la huella. Si la única holgura es <code>buffer</code> , no .
Costo de insumo +50 %	No comprar colchón. <code>calcular_requerimientos</code> ya deja un resto por redondeo de batch; ese resto es el único buffer de material.
Costo de insumo -50 %	Comprar el batch que <code>planificar</code> pida para la tanda actual, no “llenar la bodega”. El shared API (<code>API-AM0XI-500</code>) se compra cuando cualquier kit de la tanda lo necesita, no por kit.

No hay job que lea el mercado cada hora. Quien opera planta aplica la tabla. Si E2 exige autonomía 24 h, el lugar del if es `produccion.planificar` , no el portal.

D. Modelo de custodia y trazabilidad

Entregable central del diseño (§12.1 D y §8.6). Farma no conserva el historial interno del grupo: si no se escribe aquí, no se reconstruye.

El módulo **único** que escribe las tablas de custodia es `app/custodia.py` . `operaciones.py` es la fachada hacia Farma: nadie debería llamar `farma_client` desde un script de planta sin pasar por ahí.

Qué se registra y cuándo

Evento del enunciado	Cuándo	Tablas	movimientos.tipo
Recepción de un insumo comprado	Worker ve unidades nuevas en un espacio (salvo cuarentena) cuyo <code>batch</code> aún no está en <code>unidades</code>	<code>solicitudes</code> pasa a <code>recibida</code> ; <code>lotes</code> <code>origen= farma_central</code> ; <code>unidades</code> <code>en_stock</code>	<code>recepcion</code>
Generación de un producto fabricado	Igual, pero la solicitud es <code>tipo=fabricacion</code>	<code>lotes</code> <code>origen= produccion</code> ; <code>_materializar_relaciones</code>	<code>generacion</code>
Traslado entre espacios	Después de un <code>PATCH /products/{id}</code> exitoso (<code>operaciones.mover</code> o <code>worker</code> de frío)	<code>unidades.espacio_actual</code>	<code>traslado</code>
Consumo en acondicionamiento	Justo después de que Farma acepta <code>POST /products</code> de una fabricación	<code>consumos</code> (unidad, lote padre, solicitud); unidad consumida, sin espacio	<code>consumo</code>
Venta (intención)	Antes de llamar a Integrabay, al reservar FEFO	<code>ventas</code> , <code>venta_items</code> , <code>venta_unidades</code> estado <code>pendiente</code> ; unidad <code>reservada</code>	<code>venta</code>
Despacho	Tras pago <code>pagada</code> , <code>confirmar_despacho</code>	<code>venta_unidades.despachada</code> ; unidad <code>despachada</code>	<code>despacho</code>
Vencimiento	Helper <code>registrar_vencidas</code> (aún no en el loop del worker)	unidad <code>vencida</code>	<code>vencimiento</code>
Cuarentena	Enum listo; E2	—	<code>cuarentena</code>

Reglas de consistencia local (una transacción SQLAlchemy `SessionLocal.begin`):

- `registrar_fabricacion` revierte todo si falta un `_id` en custodia. No hay consumo huérfano.
- `asignar_unidades_a_venta` exige `estado == en_stock`. Si falla a mitad, la transacción no confirma.
- Llegadas son **idempotentes**: `_id` ya conocido se ignora.

- Un lote se ata a **a lo más una** solicitud (`lotes.solicitud_id` unique).

Modelo de datos

Esquema en `app/models.py` . Índices y CHECK con `naming_convention` de Alembic (migración `e1a7c0d1e2f3` : `lot_relations` , `reservada` , `venta_items`).

Tabla	Rol
<code>solicitudes</code>	Compra o fabricación pedida a Farma. <code>challenge_id</code> , <code>llega_en</code> , <code>pendiente recibida fallida</code>
<code>lotes</code>	Nacimiento conjunto. <code>codigo</code> único (el <code>batch</code> de Farma). <code>origen</code> = <code>farma_central</code> <code>produccion</code> <code>distribuidora</code> . <code>origen_grupo</code> para trueque E3
<code>unidades</code>	PK = <code>_id</code> de Farma. Estado actual y espacio
<code>consumos</code>	Unidad exacta gastada en una solicitud de fabricación. Una unidad se consume una vez
<code>lot_relations</code>	Arista N:M padre→hijo, <code>cantidad</code> , <code>solicitud_id</code> . Unique (<code>parent</code> , <code>child</code>)
<code>movimientos</code>	Bitácora por unidad (no se borra)
<code>ventas</code>	Pedido. <code>canal</code> default <code>portal</code> . <code>transaccion_id</code> Integrapay unique
<code>venta_items</code>	Líneas con <code>precio_unitario</code> vigente al confirmar el carro
<code>venta_unidades</code>	Qué <code>_id</code> se llevó cada venta; <code>pendiente</code> hasta el despacho

N:M consumido↔generado. Un lote de `BLI-AMOXI-500` tiene tres padres de insumo. Un lote de `KIT-RESP-ADULTO` tiene tres padres acondicionados. El mismo lote de `API-AMOXI-500` puede ser padre de varios blísteres. El visor recorre `lot_relations` , no asume 1:1.

Espacios (`ESPACIOS`): `checkIn` , `bodega` , `cold` , `buffer` , `packaging` , `checkOut` , `quarantine` . El worker **no** sincroniza cuarentena: un retiro no debe reaparecer como llegada.

Las cuatro preguntas (§8.6)

Implementación: `app/trazabilidad.py` . API: `GET /api/trazabilidad/{codigo}` . UI: `/trazabilidad?lote=` .

1. Unidades del lote X en inventario y dónde

- **Procedimiento:** `unidades_en_inventario(codigo)` .
- **Consulta:** `unidades` del lote con `estado IN (en_stock, reservada)` , ordenadas por espacio.
- **Mecanismo:** lookup por `lotes.codigo` (único). No recorre Farma. El visor también agrega conteos por espacio y por estado (`consultar`).

2. En qué productos se consumió X y qué lotes se generaron

- **Procedimiento:** `aguas_abajo(codigo)` .
- **Consulta:** CTE recursivo en `lot_relations` donde el lote es **padre**, profundidad < 20.
- **Mecanismo:** cada fila trae `codigo` , `sku` , `origen` , `cantidad_relacion` (unidades de ese padre que alimentaron al hijo). La UI muestra profundidad 1 aparte de la recursiva.

3. A qué clientes se entregó X, cuándo, en qué pedido

- **Procedimiento:** `clientes_del_lote(codigo)` → `_clientes` .
- **Consulta:** `ventas` ⋈ `venta_unidades` ⋈ `lotes` filtrado a `ventas.estado = 'pagada'` .
- **Mecanismo:** “se entregaron” no incluye reservas ni errores. Devuelve nombre, email, `pagada_en` , `transaccion_id` , lista de `_id` . `consultar` también arma `entregas` para el nodo verde del grafo.

4. De qué lotes de insumo se produjo X y de dónde venían

- **Procedimiento:** `aguas_arriba(codigo)` .
- **Consulta:** CTE al revés: el lote es **hijo**, se sube a padres hasta que el origen es `farma_central` O `distribuidora` (hojas del grafo).
- **Mecanismo:** cada nodo lleva `origen` y `origen_grupo` . El visor etiqueta FARMA CENTRAL / DISTRIBUIDORA n / PROPIO.

`consultar` junta las cuatro, más KPIs (`sku`, cantidad inicial, vencimiento efectivo = min `vence_en` de unidades vivas, conservación por catálogo Farma, `consulta_ms`). Medido en PROD en ~16-26 ms sobre un lote de kit.

No se barre `movimientos` entero. Eso es el requisito de rendimiento del visor.

Qué pasa si se cae a medias

Escenario del PDF: producto físicamente entregado y venta/custodia a medias.

Falla	Mitigación
Integrapay <code>init</code> falla (HTTP 5xx, URL mal armada)	<code>iniciar_checkout</code> hace <code>marcar_pago(error)</code> + <code>liberar_reserva</code> . La venta 1 de PROD quedó colgada antes de este rollback; se canceló a mano con confirmar. El rollback está en el código que corre ahora.
El cliente no vuelve de la pasarela	Unidades siguen <code>reservada</code> . No se despachan. Hay que expirar reservas (no hay TTL aún; hueco). Cancelar en Integrapay pega la <code>backUrl</code> y libera.
Pago ok, proceso muere antes de <code>confirmar_despacho</code>	<code>finalizar</code> es idempotente : si ya está <code>pagada</code> , no vuelve a despachar. El worker llama <code>recuperar_pendientes</code> cada tick.
<code>DISPATCH_FARMA=1</code> y el <code>PATCH</code> a <code>checkOut</code> falla	La fila <code>venta_unidades</code> sigue <code>pendiente</code> ; el worker reintenta. No se marca <code>despachada</code> en custodia hasta que Farma acepta y <code>confirmar_despacho</code> corre. En E1 el flag está en 0 : Farma no se entera del despacho, pero el rastro comprador↔unidad sí existe.
Farma acepta fabricación y Postgres no guarda consumos	Log <code>CRITICAL</code> con sku, challenge, <code>_id</code> . No hay XA. Planta corrige a mano; no se sigue fabricando a ciegas (candado).
Dos fabricaciones eligen las mismas unidades	<code>pg_try_advisory_lock(4000001)</code> . La segunda espera o falla explícito.
Lote llega sin solicitud pendiente	<code>registrar_llegadas</code> no crea lote huérfano; log de error.

El visor, si el lote no está, responde 404. No inventa genealogía.

Logística de operación y huecos (Entrega 1)

Esta sección no sustituye A-D; deja por escrito cómo está levantado el grupo 4 y qué queda fuera del software.

Servidor y URL

El enunciado asigna `https://distribuidorX.ing.uc.cl`. El hostname `distribuidor4.ing.uc.cl` **no resuelve en DNS** (comprobado). El servicio público que corre es:

<https://choripan4.ing.uc.cl>

Ruta	Qué es
/	Portal de venta (catálogo)
/carrito, /checkout, /pago/{exito error cancelado}	Compra e Integrapay <code>backUrls</code>
/trazabilidad	Visor (query <code>?lote=</code>)
/pedidos	Bitácora local de ventas
/api/...	JSON
/health	Postgres

TLS termina en Nginx del host; proxea a `127.0.0.1:3000` (Uvicorn). Ejemplo de config: `deploy/nginx.example.conf` . `PUBLIC_BASE_URL=https://choripan4.ing.uc.cl` para que Integrapay redirija al HTTPS real.

SSH: usuario `integracion` en ese host. Firewall del curso: no tocar `ufw` . Disco de Postgres: volumen Docker `pgdata` . **Prohibido** `docker compose down -v` .

RAM: db 512m, web 512m, worker 256m / 1 CPU. Al redespregar se para el worker, se buildea `web` y se vuelve a subir; no se recrea `db` .

Código y Canvas

- **GitHub:** todo el código en el repositorio asignado al grupo. Rama de trabajo reciente: `deploy/entregal` sobre lo que ya estaba en `origin/main` .
- **Canvas:** este informe (PDF generado desde `INFORME.md`). El Markdown en el repo no reemplaza la tarea de Canvas.
- Invitación al cuerpo docente al repo: proceso del curso (el staff invita al grupo). Si piden acceso extra, se gestiona aparte; no es un entregable de este directorio.

Treinta kits

Cumplido en PROD el 6-7 de octubre de 2026. `logs.txt` arranca la campaña de 10 SKU × 30. Ejemplos de lote: fabricación #56 = 6 × `KIT-RESP-ADULT0` ; más tarde el portal vendió `L-KIT-RESP-ADULT0-261006-60e8` (pedido #6) y `L-KIT-DERMATO-261007-0c20` (pedido #4).

`sembrar_demo` (`L-DEMO-*`) **no** se corre en PROD: ensuciaría genealogía con lotes ficticios.

Pagos y Farma en el ambiente evaluado

Variable	Valor en PROD al cerrar E1	Efecto
CHECKOUT_MODE	real	Integrapay de verdad, ~20 % error simulado por ellos
DISPATCH_FARMA	0	Despacho solo en custodia
FARMA_ENV	prod	Espacios y catálogo de producción
CHECKOUT_BASE_URL	host prod.proyecto.2026-2... sin /checkout	auth + init

En local el default es `CHECKOUT_MODE=mock` (`.env.example`).

Huecos conscientes (no se venden como hechos)

1. Job de vencimiento en el worker (`registrar_vencidas` existe, no está en el tick).
2. TTL de reservas si el cliente abandona Integrapay.
3. `POST /api/checkout/confirmar` sin autenticar al comprador (confía en la `backUrl`).
4. Política ± 50 % documentada, no cron.
5. DNS `distribuidor4.ing.uc.cl` .
6. Despacho físico Farma apagado.
7. `02_arquitectura_...md` describe un stack que **no** se implementó; el diseño válido es este directorio.

Nada de eso impide cumplir el checklist de E1 (portal, visor, 30 kits, HTTPS, custodia, informe). Sí condiciona E2 si hay recall o autonomía 48 h.

Declaración de uso de inteligencia artificial

Obligatoria en el informe (PDF Entrega 1, p. 18; enunciado §18). El mismo contenido, resumido, está en el `README.md` del proyecto.

El uso de IA está permitido. El cuerpo docente puede pedir a cualquier integrante explicar oralmente custodia, PoW o checkout. No entender lo entregado es problema de integridad académica, no de la herramienta.

Herramientas y modelos

Herramienta	Para qué
Cursor (agente) con modelo Grok	Implementación del portal/visor/pagos, este informe, revisión contra el PDF
Graphify (<code>user-graphify</code> , grafo en <code>graphify-out/</code>)	Mapa de componentes y caminos abastecimiento → Farma → custodia, venta → Integrapay
Codebase Memory MCP (proyecto <code>integracion</code>)	Símbolos, <code>trace_path</code> , rutas HTTP, outline de <code>custodia.py</code> / <code>ventas.py</code>
Playwright (Chromium)	Click-through de catálogo, pago, visor con <code>?lote=</code> , pedidos
Documentación OpenAPI del curso (Farma e Integrapay)	Contratos <code>auth</code> / <code>init</code> / <code>challenge</code> / <code>products</code> ; no se copió prosa del enunciado como si fuera código

No se usó un modelo para “inventar” recetas de kits: las cantidades salen de Farma (`components` , `production.batch`) y del ejemplo §4 del enunciado. Los números de solicitudes (#1, #32, #56, venta #6) salen de `logs.txt` y de la API de ventas en PROD.

Tareas asignadas a la IA

1. Leer el enunciado y la propuesta `02` , contrastarlos con el repo (Python ya integraba Farma) y **no** migrar a Next.js.
2. Completar custodia N:M (`lot_relations`), portal Vite, adapter Integrapay, visor con CTE, worker de frío.
3. Despliegue cuidado en la VM (backup, Alembic, sin `down -v`).
4. Corregir UX post-pago (`/trazabilidad?lote=`), bitácora `/pedidos` , botones con caja.
5. Redactar este informe: diagramas instanciados, decisiones con pérdida explícita, estrategia con tanda/batch, modelo D, logística incluyendo el DNS que no resuelve.

La IA **no** eligió secretos, no corrió `DISPATCH_FARMA=1` , no sembró demo en PROD y no reescribió el enunciado.

Cómo se verificó

Qué	Cómo
PoW	<code>tests/test_checkout_pow.py</code> contra la regla <code>sha256(prefix:nonce)</code> y leading-zero bits del anexo
Payload Integrapay	El mismo test arma <code>backUrls</code> ; en PROD se vio <code>redirect_url</code> real y el ~20 % de error
Custodia / visor	Consultar un lote <code>L-KIT-*</code> producido en la campaña; tiempo de respuesta del orden de 20 ms; aguas arriba llegan a Farma
Portal	Playwright: catálogo con precio/stock, checkout, vuelta <code>exito</code> , botón de trazabilidad con código de lote
Diagramas de este informe	Actores y mensajes cruzados con Graphify/ <code>trace_path</code> (<code>comprar</code> , <code>fabricar</code> , <code>iniciar_checkout</code> , <code>consultar</code>) y con líneas de <code>logs.txt</code>
Requisitos vs. propuesta 02	Si 02 y el código discrepan, gana el enunciado y el código que corre

Criterio de parada: un integrante del grupo tiene que poder dibujar en pizarra (1) la reserva FEFO antes de Integrapay, (2) por qué el visor no pega a Farma, (3) por qué el PoW no va en el worker HTTP de Uvicorn.
